

Homework #3 - Simple DL Workflow in GKE

COMS 6998: Applied Machine Learning in the Cloud

Dakshinamoorthy A (da3232)
Columbia University

May 8, 2026

1 Objective

The objective of this assignment is to develop Container and Kubernetes artifacts to perform Deep Learning (DL) training and DL inference hosting in Google Kubernetes Engine (GKE). This involves creating Dockerfiles, Kubernetes manifests (Jobs, Deployments, Services), and managing persistent storage (PVC) to handle model state across different phases of the lifecycle.

2 Architecture Aspects & Requirements

The project implements a simple yet representative DL workload: ****MNIST Digit Classification**** using PyTorch. The architecture consists of two primary phases:

2.1 Training Phase

The training phase is implemented using a **Kubernetes Job**.

- **Why Job?:** Training is a finite task with a clear start and end. The Job controller ensures the pod runs until the script completes and the model is saved.
- **Hardware:** The Job requests 1 NVIDIA Tesla T4 GPU to accelerate the training process.

2.2 Inference Phase

The inference phase is implemented using a **Kubernetes Deployment** and a **LoadBalancer Service**.

- **Why Deployment?:** Inference requires high availability and scaling capabilities. The Deployment controller ensures the web server is always running.
- **Interactive UI:** A Flask-based web server provides a canvas for users to draw digits and receive real-time predictions.

3 Implementation Details

3.1 Infrastructure Provisioning

The cluster was created as a Private GKE Cluster in the **asia-east1-c** zone to comply with project-level security constraints regarding external IP addresses.

```

da3232@assign-3:~/AMLC_Assign_3$ gcloud container clusters create mnist-cluster \
--num-nodes=1 \
--machine-type=n1-standard-4 \
--accelerator=type=nvidia-tesla-t4,count=1,gpu-driver-version=default \
--zone=asia-east1-c \
--enable-private-nodes \
--master-ipv4-cidr=172.16.0.48/28 \
--enable-ip-alias
Note: Machines with GPUs have certain limitations which may affect your workflow. Learn more at https://cloud.google.com/kubernetes-engine/docs/how-to/gpus
Note: Starting in GKE 1.30.1-gke.115600, if you don't specify a driver version, GKE installs the default GPU driver for your node's GKE version.
Creating cluster mnist-cluster in asia-east1-c... Cluster is being health-checked (Kubernetes Control Plane is healthy)...done.
Created [https://container.googleapis.com/v1/projects/coms6998-amlc-da3232/zones/asia-east1-c/clusters/mnist-cluster].
To inspect the contents of your cluster, go to: https://console.cloud.google.com/kubernetes/workload/_gcloud/asia-east1-c/mnist-cluster?project=coms6998-amlc-da3232
kubeconfig entry generated for mnist-cluster.
NAME          LOCATION    MASTER_VERSION  MASTER_IP      MACHINE_TYPE  NODE_VERSION    NUM_NODES  STATUS
mnist-cluster asia-east1-c 1.35.3-gke.1389000 104.199.195.0  n1-standard-4 1.35.3-gke.1389000 1           R
UNNING  IPV4
da3232@assign-3:~/AMLC_Assign_3$ gcloud container clusters get-credentials mnist-cluster --zone=asia-east1-c --internal-ip
Fetching cluster endpoint and auth data.
kubeconfig entry generated for mnist-cluster.
da3232@assign-3:~/AMLC_Assign_3$

```

Figure 1: GKE Cluster Creation in asia-east1-c

```

kubeconfig entry generated for mnist-cluster.
da3232@assign-3:~/AMLC_Assign_3$ kubectl get nodes
kubectl describe nodes | grep -i gpu
NAME          STATUS    ROLES    AGE      VERSION
gke-mnist-cluster-default-pool-e075f6e9-9t1v Ready    <none>   2m48s    v1.35.3-gke.1389000
cloud.google.com/gke-gpu-driver-version=default
nvidia.com/gpu: 1
nvidia.com/gpu: 1
kube-system      nvidia-gpu-device-plugin-small-cos-174fl 150m (3%) 1 (25%)
80Mi (0%) 115Mi (0%) 2m47s
nvidia.com/gpu 0
da3232@assign-3:~/AMLC_Assign_3$

```

Figure 2: Verification of Nodes and GPU Availability

3.2 Containerization

Two separate Dockerfiles were created for training and inference. The images were built and pushed to the Google Artifact Registry.

```

da3232@assign-3:~/AMLC_Assign_3$ docker build -t asia-east1-docker.pkg.dev/coms6998-amlc-da3232/ml-images/mnist
--train:latest -f Dockerfile.train .
docker push asia-east1-docker.pkg.dev/coms6998-amlc-da3232/ml-images/mnist-train:latest
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
Install the buildx component to build images with BuildKit:
https://docs.docker.com/go/buildx/

Sending build context to Docker daemon 969.7kB
Step 1/6 : FROM pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime
2.1.0-cuda12.1-cudnn8-runtime: Pulling from pytorch/pytorch
f030756bea58: Pulling fs layer
4f4fb700ef54: Pulling fs layer
7007490126ef: Pulling fs layer
e991862a3134: Pulling fs layer
002e0840847c: Pulling fs layer
4f4fb700ef54: Download complete
e991862a3134: Download complete
7007490126ef: Download complete
f030756bea58: Download complete
7007490126ef: Pull complete
f030756bea58: Pull complete
002e0840847c: Download complete
002e0840847c: Pull complete
4f4fb700ef54: Pull complete
e991862a3134: Pull complete
Digest: sha256:e4aaefef0c96318759160ff971b527ae61ae306a1204c5f6e907c4b45f05b8a3
Status: Downloaded newer image for pytorch/pytorch:2.1.0-cuda12.1-cudnn8-runtime
--> e4aaefef0c96
Step 2/6 : WORKDIR /app
--> Running in e97d3e07f08b
--> Removed intermediate container e97d3e07f08b
--> 7b72ddc173ae

```

Figure 3: Building and Pushing Docker Images (Part 1)

```

--> 709713a022f2
Successfully built 709713a022f2
Successfully tagged asia-east1-docker.pkg.dev/coms6998-amlc-da3232/ml-images/mnist-train:latest
The push refers to repository [asia-east1-docker.pkg.dev/coms6998-amlc-da3232/ml-images/mnist-train]
a63e44fb00b8: Pushed
5b9de28d59f0: Pushed
7007490126ef: Pushed
f030756bea58: Pushed
002e0840847c: Pushed
4fffb700ef54: Layer already exists
e991862a3134: Pushed
5f93595ae798: Pushed
latest: digest: sha256:709713a022f2098792efa07a6822d744f5f20de8e1080dcbeadb4df95f5a59de size: 2076

```

Figure 4: Building and Pushing Docker Images (Part 2)

3.3 Persistent Storage (PVC)

A Persistent Volume Claim was created to store the trained model. This allows the inference pod to load the model saved by the training pod.

```

^[[^Cda3232@assign-3:~/AMLC_Assign3$ kubectl get pvc pvc-
NAME          STATUS    VOLUME          CAPACITY   ACCESS MODES   STORAGECLASS   VOLUME
ATTRIBUTESCLASS AGE
model-pvc     Bound     pvc-85c4bdb6-74f9-45c8-a4b6-0c4fa0c9a654 1Gi         RWO             standard-rwo   <unset>
>
da3232@assign-3:~/AMLC_Assign_3$

```

Figure 5: Verification of Bound Persistent Volume Claim

4 Execution and Results

4.1 DL Training

The training job was executed using the GPU. The logs show the model successfully iterating through epochs and saving the result to the PVC.

```

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz
Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-images-idx3-ubyte.gz to ./data/MNIST/raw/t10k-images-idx3-ubyte.gz
100%|██████████| 1648877/1648877 [00:01<00:00, 1253823.33it/s]
Extracting ./data/MNIST/raw/t10k-images-idx3-ubyte.gz to ./data/MNIST/raw

Downloading http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz
Failed to download (trying next):
HTTP Error 404: Not Found

Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz
Downloading https://oss-ci-datasets.s3.amazonaws.com/mnist/t10k-labels-idx1-ubyte.gz to ./data/MNIST/raw/t10k-labels-idx1-ubyte.gz
100%|██████████| 4542/4542 [00:00<00:00, 2141230.61it/s]
Extracting ./data/MNIST/raw/t10k-labels-idx1-ubyte.gz to ./data/MNIST/raw

Using device: cuda
Train Epoch: 1 [0/60000 (0%)] Loss: 2.319517
Train Epoch: 1 [6400/60000 (11%)] Loss: 1.088659
Train Epoch: 1 [12800/60000 (21%)] Loss: 0.569277
Train Epoch: 1 [19200/60000 (32%)] Loss: 0.528796
Train Epoch: 1 [25600/60000 (43%)] Loss: 0.469504
Train Epoch: 1 [32000/60000 (53%)] Loss: 0.343877
Train Epoch: 1 [38400/60000 (64%)] Loss: 0.344105
Train Epoch: 1 [44800/60000 (75%)] Loss: 0.371195
Train Epoch: 1 [51200/60000 (85%)] Loss: 0.468099
Train Epoch: 1 [57600/60000 (96%)] Loss: 0.441965
Train Epoch: 2 [0/60000 (0%)] Loss: 0.152205
Train Epoch: 2 [6400/60000 (11%)] Loss: 0.200550
Train Epoch: 2 [12800/60000 (21%)] Loss: 0.233512
Train Epoch: 2 [19200/60000 (32%)] Loss: 0.511238
Train Epoch: 2 [25600/60000 (43%)] Loss: 0.381454
Train Epoch: 2 [32000/60000 (53%)] Loss: 0.187599
Train Epoch: 2 [38400/60000 (64%)] Loss: 0.189037
Train Epoch: 2 [44800/60000 (75%)] Loss: 0.201518
Train Epoch: 2 [51200/60000 (85%)] Loss: 0.212293
Train Epoch: 2 [57600/60000 (96%)] Loss: 0.218335
Model saved to /mnt/model/model.pth
da3232@assign-3:~/AMLC_Assign_3$

```

Figure 6: Training Logs showing CUDA usage and Model Saving

4.2 Interactive Inference

The inference engine was exposed via a LoadBalancer. The following screenshots demonstrate the service creation and the interactive UI predicting a hand-drawn digit.

```
da3232@assign-3:~/AMLC_Assign_3$ ^C
da3232@assign-3:~/AMLC_Assign_3$ kubectl get service mnist-service -w
NAME                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
mnist-service       LoadBalancer  34.118.239.43  34.80.13.20    80:31336/TCP     100s
^
```

Figure 7: Service Creation and External IP Assignment

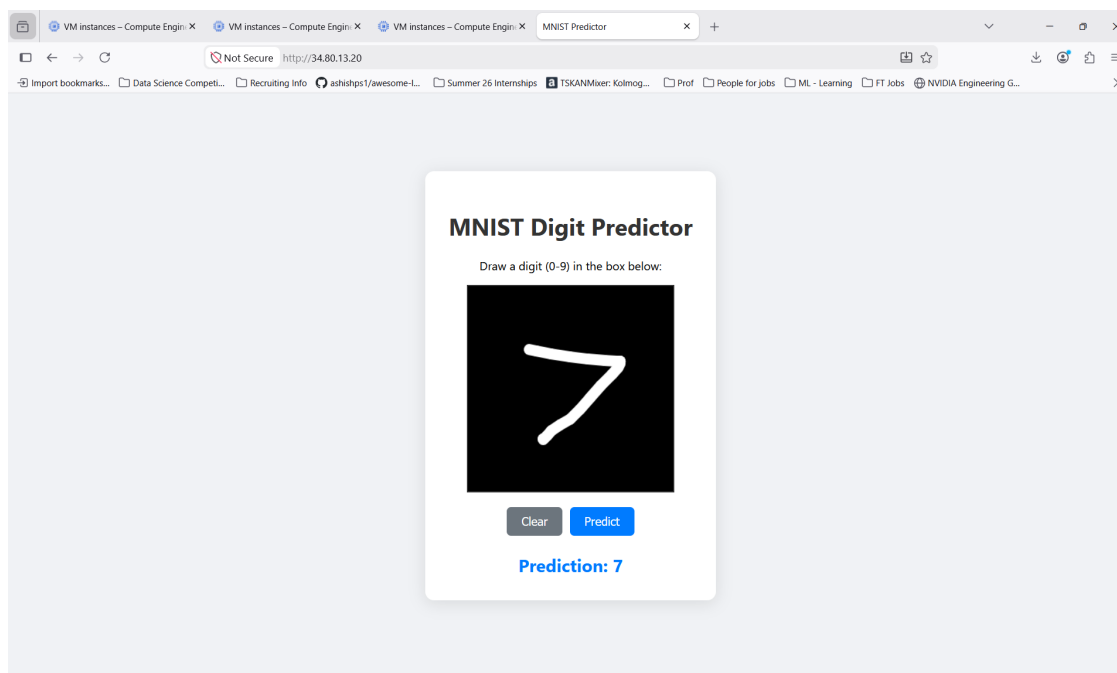


Figure 8: Interactive MNIST Predictor UI

```
service/mnist-service created
da3232@assign-3:~/AMLC_Assign_3$ kubectl get pods
NAME                                READY   STATUS             RESTARTS   AGE
mnist-inference-84c55db7bc-m8785    0/1     ContainerCreating   0          10s
mnist-train-8d9ww                   0/1     Completed           0          5m5s
da3232@assign-3:~/AMLC_Assign_3$
```

Figure 9: Status of Kubernetes Pods after Deployment

5 Experience and Reflection

The primary challenge in this assignment was navigating the specific regional resource availability (GCE Stockouts for GPUs) and security constraints (External IP violations). Implementing a Private GKE Cluster provided a secure and robust solution. The use of native Kubernetes objects (Jobs and Deployments) allowed for a clear separation of concerns between the transient training process and the persistent serving process.